

Angular e il progetto AVANT: Secure by design

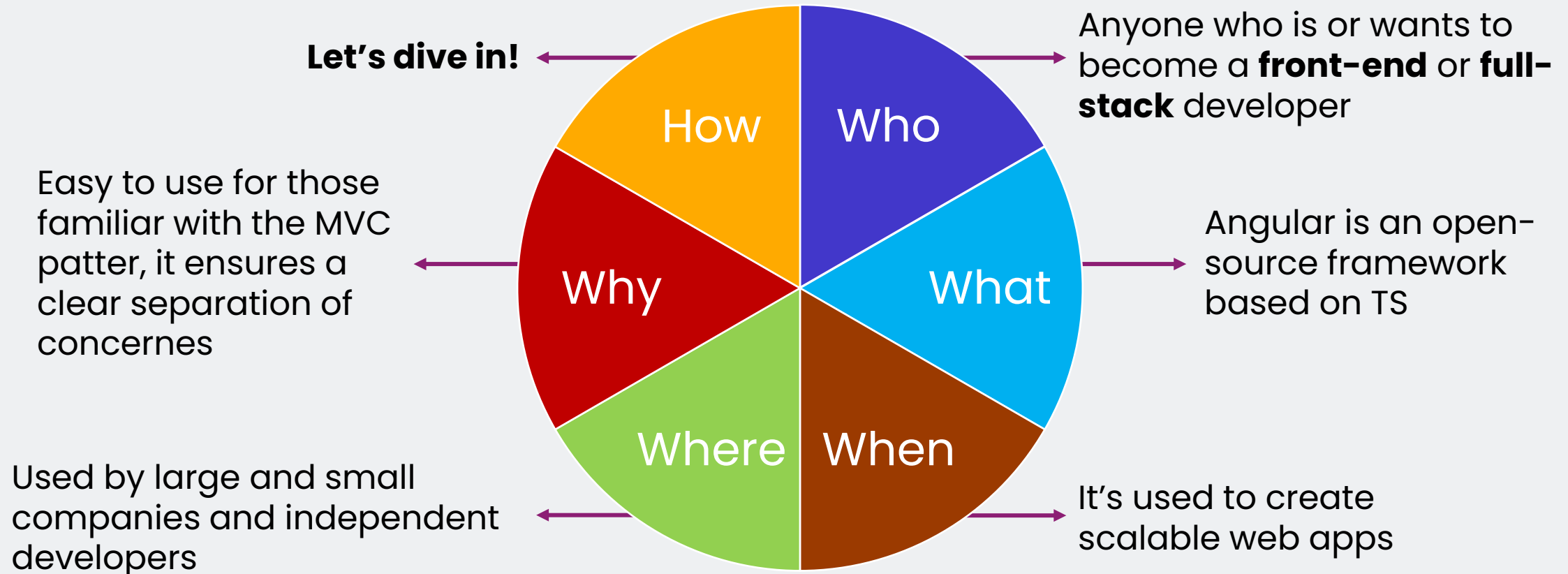
Introduction to Angular and the
application of its key principles in the
AVANT project

04/05/26

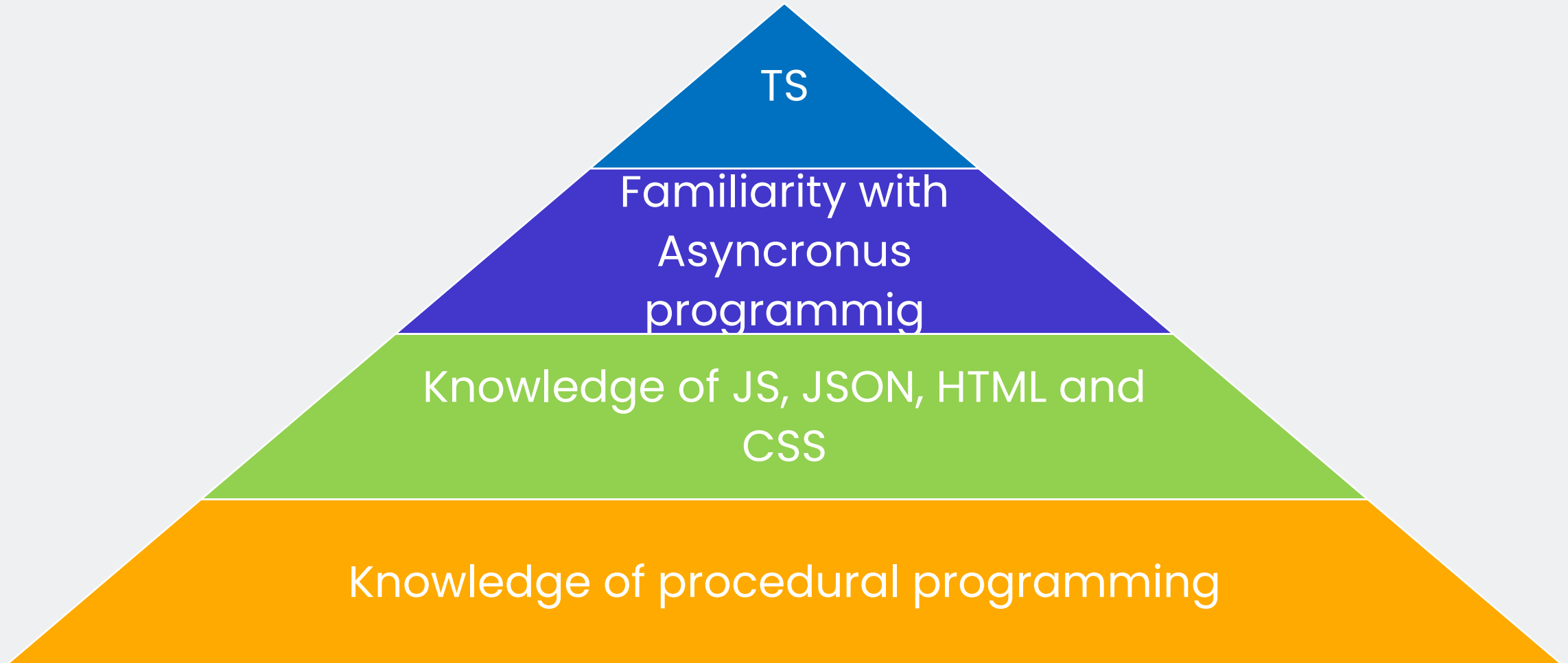
AGENDA

- Talking about Angular and his environment
- Talking about AVANT project

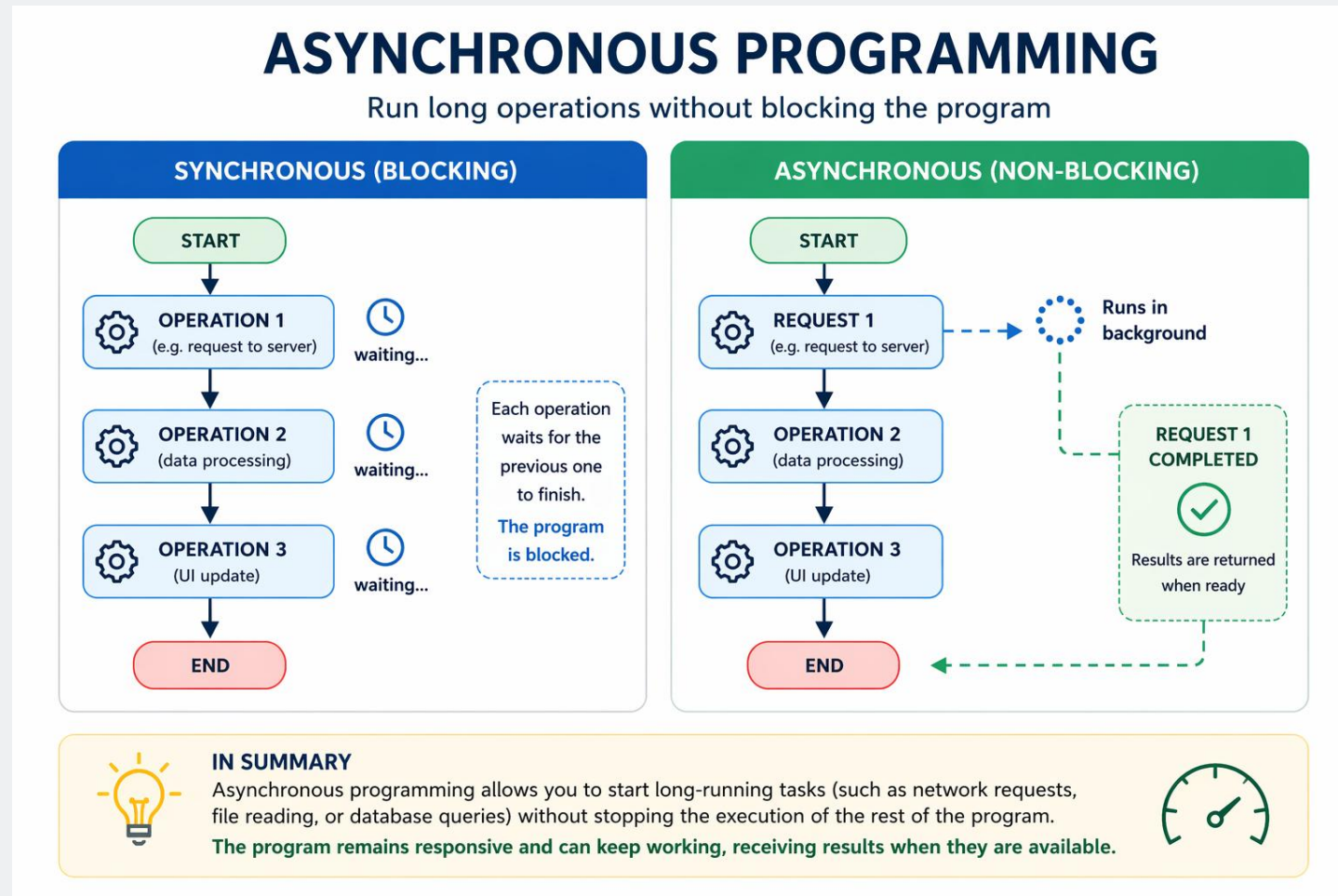
5 questions about Angular



Foundamentals of Angular



What is asynchronous programming



What is TypeScript

TS

INTRODUCTION TO TYPESCRIPT

TypeScript is a superset of JavaScript that adds static typing and other features to write safer, more maintainable code.

JS

TYPESCRIPT		JAVASCRIPT
<pre>1 let age: number = 25; 2 let name: string = "Alice"; 3 let isActive: boolean = true; 4 let scores: number[] = [10, 20, 30];</pre>	TYPES TypeScript adds static types to variables, function parameters, and return values.	<pre>1 let age = 25; 2 let name = "Alice"; 3 let isActive = true; 4 let scores = [10, 20, 30];</pre>
<pre>1 function add(a: number, b: number): number { 2 return a + b; 3 } 4 add(2, 3); // ✔ OK 5 add("2", 3); // ✖ Error: Argument of 6 // type 'string' is not // assignable to parameter // of type 'number'.</pre>	SAFETY TypeScript catches errors at compile time, before your code runs.	<pre>1 function add(a, b) { 2 return a + b; 3 } 4 add(2, 3); // ✔ OK 5 add("2", 3); // ⚠ No error (may cause bugs)</pre>
✓ Autocompletion ✓ Better refactoring ✓ Interfaces & Enums ✓ Generics ✓ Modern OOP features ...and more!	TOOLING Better tooling and developer experience out of the box.	⚠ No types ⚠ No compile-time checks ⚠ Fewer built-in tools ...you rely on runtime errors

**IN SUMMARY**

TypeScript helps you write JavaScript code with types. It improves code quality, reduces bugs, and scales better for large projects and team development.

.ts
TypeScript Code

→

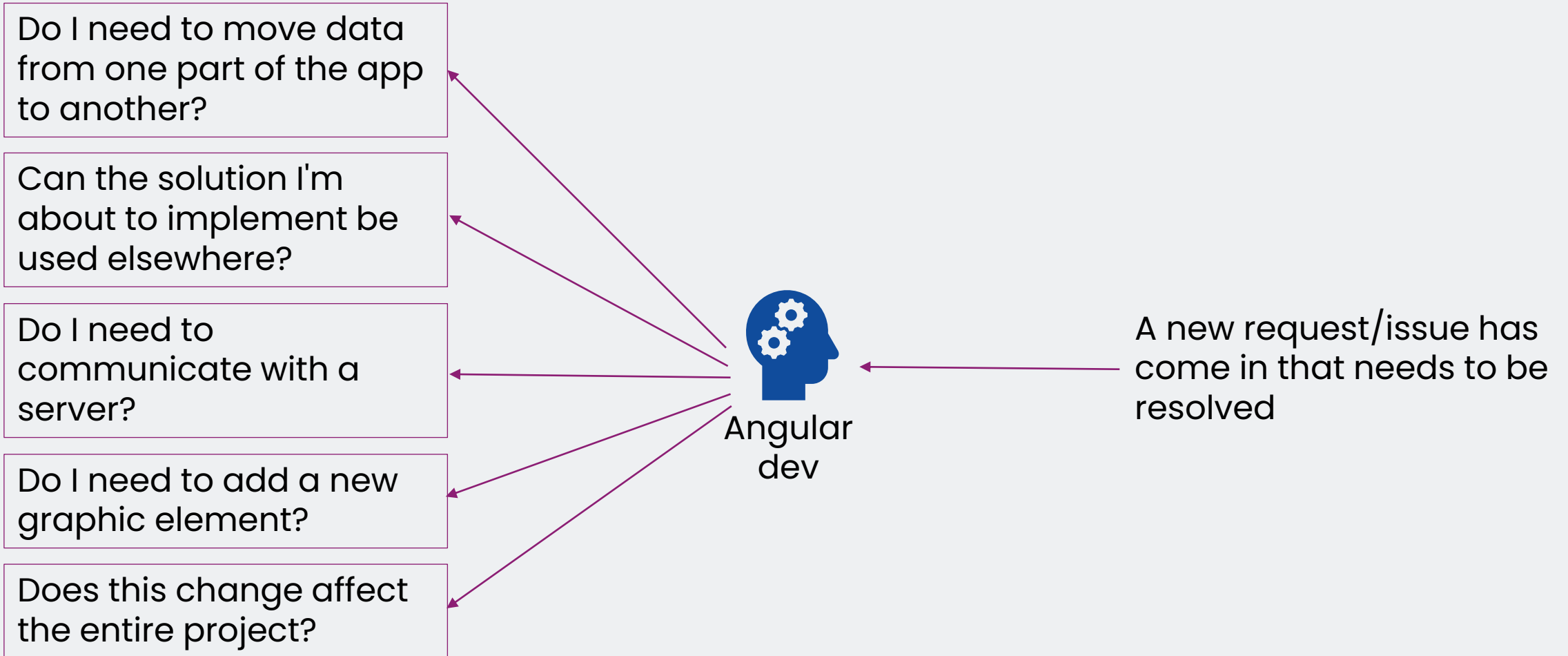
Compiler (tsc)

→

.js
JavaScript Output

TypeScript code is compiled to plain JavaScript that runs in any browser or JavaScript engine.

How does an Angular developer think?



What is an Angular component?



ANGULAR COMPONENT

The building block of every Angular application



A component is a TypeScript class decorated with `@Component`. It controls a piece of the UI and its behavior.

WHAT IS A COMPONENT?

-  Encapsulates a part of the user interface (HTML + CSS + Logic)
-  Reused and composed to build complex UIs
-  Manages its own data and interacts with other components and services

ANATOMY OF A COMPONENT

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello',
  templateUrl: './hello.component.html',
  styleUrls: ['./hello.component.css']
})
export class HelloComponent {
  title = 'Hello Angular!';
}
```

- SELECTOR**
The HTML tag used to insert the component.
- TEMPLATE**
The HTML that defines the component's view.
- STYLES**
The CSS that styles the component.
- CLASS**
The TypeScript class that contains data and logic.

EXAMPLE: HELLO COMPONENT

HTML (Template)

```
<h2>{{ title }}</h2>
<p>This is my first
Angular component!</p>
```

CSS (Styles)

```
h2 {
  color: #e53935;
  font-weight: bold;
}
p { color: #555; }
```

TypeScript (Logic)

```
export class HelloComponent {
  title = 'Hello Angular!';
}
```

RESULT IN THE BROWSER

Hello Angular!
This is my first Angular component!

HOW TO USE IT

Insert the selector in any template:

```
<app-hello></app-hello>
```

↓

The component will be rendered in that place.

KEY TAKEAWAY

Components let you break the UI into independent, reusable pieces.
Everything in Angular is a component!

COMPONENT TREE

```
graph TD
  AppComponent[AppComponent] --> HeaderComponent[HeaderComponent]
  AppComponent --> MainComponent[MainComponent]
  AppComponent --> FooterComponent[FooterComponent]
  MainComponent --> ListComponent[ListComponent]
  MainComponent --> ItemComponent[ItemComponent]
```

BENEFITS

- ✓ Reusability
- ✓ Maintainability
- ✓ Scalability
- ✓ Testability

How I can transform my data?



ANGULAR PIPES

Transform data in your templates



A pipe in Angular is a simple way to **transform data** in a template. It takes input, processes it, and returns the transformed output.

WHAT IS A PIPE?

-  Pipes transform data for display in templates.
-  They are simple, reusable and declarative.
-  Angular provides many built-in pipes.
-  You can also create your own custom pipes.

SYNTAX

```
{{ value | pipeName [: parameter1 [: parameter2 ...]] }}
```

-  **value**
The data to be transformed.
-  **pipeName**
The name of the pipe to apply.
-  **parameter(s)**
Optional values to customize the transformation.

HOW IT WORKS



The pipe receives the input, transforms it, and returns the result to be displayed.

BUILT-IN PIPES EXAMPLES

PIPE	DESCRIPTION	EXAMPLE	RESULT
<code>uppercase</code>	Converts text to uppercase	<code>{{ 'hello' uppercase }}</code>	HELLO
<code>date</code>	Formats a date	<code>{{ today date:'dd/MM/yyyy' }}</code>	24/05/2024
<code>currency</code>	Formats a number as currency	<code>{{ 1234.5 currency:'EUR':'symbol':'1.2-2' }}</code>	€1,234.50
<code>percent</code>	Formats a number as percent	<code>{{ 0.25 percent:'1.0-1' }}</code>	25.0%
<code>slice</code>	Returns a part of an array or string	<code>{{ [1,2,3,4,5] slice:1:4 }}</code>	[2, 3, 4]

CUSTOM PIPE EXAMPLE

- Create the pipe (TypeScript)

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'exclaim'
})
export class ExclaimPipe implements PipeTransform {
  transform(value: string): string {
    return value + '!';
  }
}
```
- Use it in a template (HTML)

```
<p>{{ 'Angular is awesome' | exclaim }}</p>
```



```
<p>Angular is awesome!</p>
```

BENEFITS

-  Keep templates clean and readable
-  Promote reusability and consistency
-  Encapsulate presentation logic, not business logic
-  Easy to test and maintain

KEY TAKEAWAY

Pipes are a powerful feature in Angular to transform and format data directly in templates in a simple and efficient way.

How the components share the data?


ANGULAR

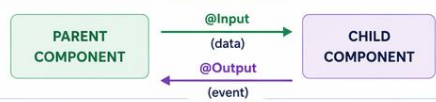
INPUT & OUTPUT BETWEEN COMPONENTS

How components communicate with each other



Angular components communicate using **@Input** to pass data from a parent to a child, and **@Output** to send events from a child to a parent.

THE BIG PICTURE





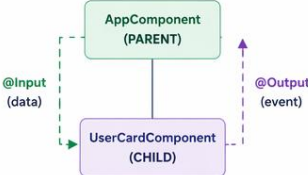
@INPUT – Parent to Child

Use **@Input()** to send data from the parent component to a child component.

```
@Input() propertyName: Type;
```

Example: The parent sends a user object to the child.

COMPONENT TREE





@OUTPUT – Child to Parent

Use **@Output()** with **EventEmitter** to send events from the child component to the parent.

```
@Output() eventName = new EventEmitter<Type>();
```

Example: The child notifies the parent about an action (e.g., a button click).

1. PASS DATA FROM PARENT TO CHILD (@INPUT)

```
parent.component.ts
```

```

user = {
  id: 1,
  name: 'Alice',
  role: 'Admin'
};

```

```
child.component.ts
```

```

import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.component.html'
})
export class UserCardComponent {
  @Input() user!: { id: number; name: string; role: string };
}

```

parent.component.html

```
<app-user-card [user]="user"></app-user-card>
```

Result in Child Component

Receives 'user' object and can display it in its template.

2. SEND EVENTS FROM CHILD TO PARENT (@OUTPUT)

```
child.component.ts
```

```

import { Component, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-user-card',
  template: `
    <button (click)="notify()">Delete</button>
  `
})
export class UserCardComponent {
  @Output() deleted = new EventEmitter<number>();
  notify() {
    this.deleted.emit(this.user.id);
  }
}

```

```
parent.component.ts
```

```

onUserDeleted(id: number) {
  console.log('User deleted:', id);
  // update list, show message, etc.
}

```

```
parent.component.html
```

```

<app-user-card
  [user]="user"
  (deleted)="onUserDeleted($event)"
</app-user-card>

```

Result in Parent Component

The parent handles the event and executes its logic.

KEY TAKEAWAY



Use **@Input()** to pass data down the component tree.
Use **@Output()** to send events up the component tree.



Promotes reusable and independent components



Keeps data flow predictable and unidirectional



Improves maintainability and makes components easier to test

How I can improve the separation of concerns?



ANGULAR SERVICES

Sharing data and logic across your application



A Service in Angular is a class with a specific purpose. It contains reusable business logic, data access, or shared functionality that can be used by multiple components.



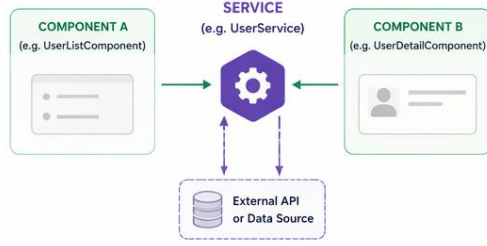
KEY IDEA

Services help you keep components focused on the UI, not on the logic.

WHY USE SERVICES?

- REUSABILITY**
Write once, use everywhere.
- SEPARATION OF CONCERNS**
Keep components clean and focused on presentation.
- SHARING DATA**
Share data or state between multiple components.
- TESTABILITY**
Easier to test and maintain.

HOW IT WORKS



Components inject the service and call its methods. The service handles the logic and data, and can be shared by many components.

EXAMPLE: USER SERVICE

user.service.ts

```
@Injectable({
  providedIn: 'root' // Singleton: one instance for the whole app
})
export class UserService {
  private apiUrl = 'https://api.example.com/users';
  constructor(private http: HttpClient) {}

  // Get all users
  getUsers(): Observable<User[]> {
    return this.http.get<User[]>(this.apiUrl);
  }

  // Get a user by id
  getUser(id: number): Observable<User> {
    return this.http.get<User>(`${this.apiUrl}/${id}`);
  }
}
```

HOW TO USE A SERVICE IN A COMPONENT

- Import the service
- Inject it in the constructor
- Use its methods



user-list.component.ts

```
export class UserListComponent implements OnInit {
  users: User[] = [];

  constructor(private userService: UserService) {} // Inject the service

  ngOnInit(): void {
    this.userService.getUsers().subscribe(data => {
      this.users = data;
    });
  }
}
```

SERVICE PROVIDER

Services are provided using `@Injectable` with `providedIn`.

providedIn	Meaning
'root'	One instance for the entire app (recommended)
'any'	A new instance in each lazy-loaded module
null	You must provide the service manually in a module or component

BUILT-IN SERVICES EXAMPLES

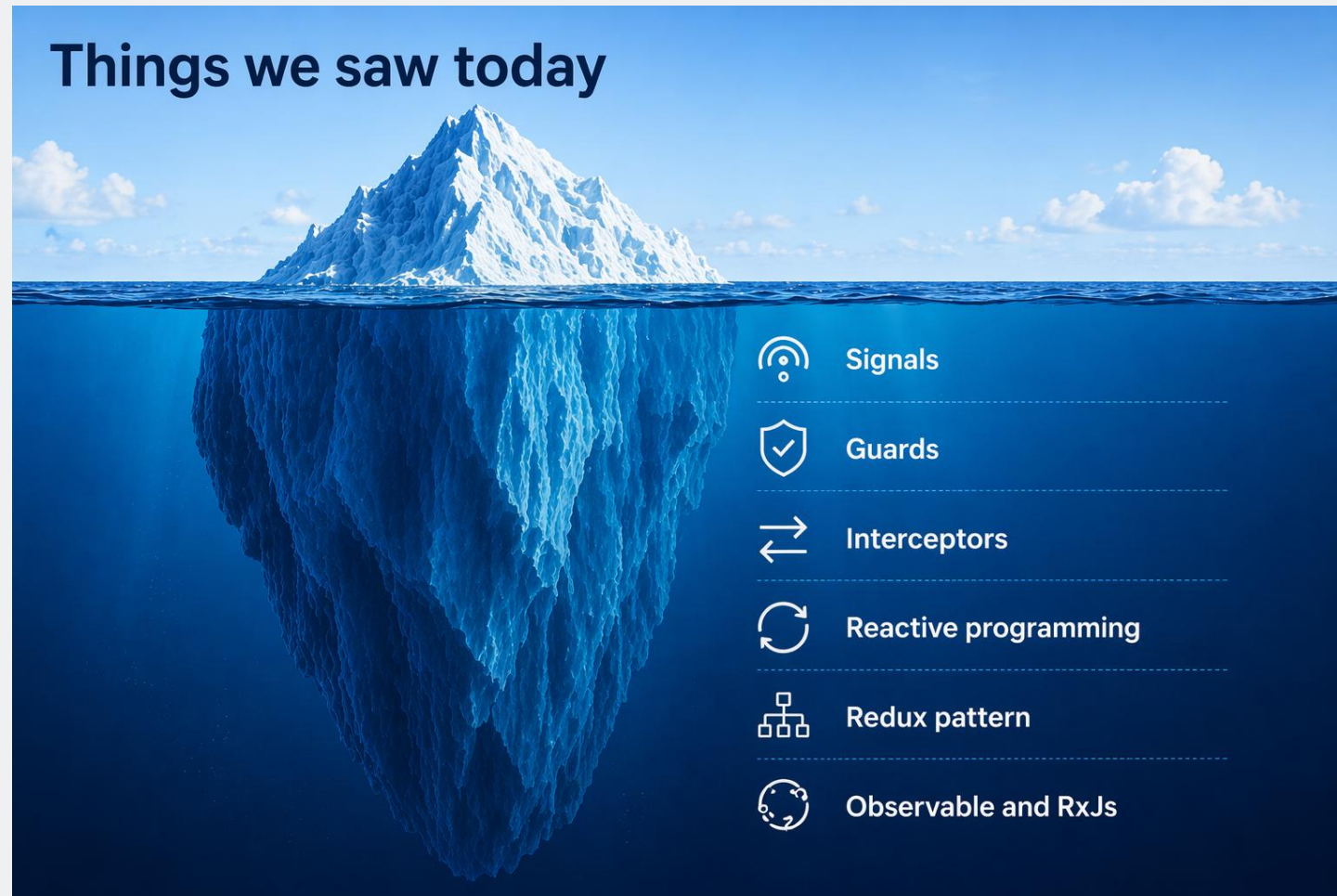
-  **HttpClient**
Make HTTP requests to a backend API.
-  **Router**
Navigate between views.
-  **LocalStorage / SessionStorage**
Store data in the browser.
-  **MatSnackBar (Angular Material)**
Show messages to the user.



KEY TAKEAWAY

Services are the backbone of an Angular application. They promote code reuse, maintainability, and a clean architecture by separating business logic and data management from the UI.

Our iceberg



Put it all together

- Let's see an example of an Angular app for a Coin Collector
- <https://github.com/CiccioTecchio/My-Aws-Coins-Collections>
- (remember to star the repo! 🌟)

Using this concept we have developed RATING-DT

- A cyber-security tool that can assess the risks of a business service
- It can give information and analytics about how it's asset are vulnerable and risks exposed
- And now let's see how it works with a **live demo!**



Francesco Vicidomini

francesco.vicidomini@eng.it

Technical Researcher Specialist